# 致读者：

我从 2002 年 7 月开始翻译这本书，当时还是第二版。但是翻完前言和介绍部分后，chinapub 就登出广告，说要出版侯捷的译本。于是我中止了翻译，等着侯先生的作品。

我是第一时间买的 这本书，但是我失望了。比起第一版，我终于能看懂这本书了，但是相比我的预期，它还是差一点。所以当 Bruce Eckel 在他的网站上公开本书的第三版的时候，我决定把它翻译出来。

说说容易，做做难。一本 1000 多页的书不是那么容易翻的。期间我也曾打过退堂鼓，但最终还是全部翻译出来了。从今年的两月初起，到 7 月底，我几乎放弃了所有的业余时间，全身心地投入本书的翻译之中。应该说，这项工作的难度超出了我的想像。

首先，读一本书和翻译一本书完全是两码事。英语与中文是两种不同的语言，用英语说得很畅的句子，翻成中文之后就完全破了相。有时我得花好几分钟，用中文重述一句我能用几秒钟读懂的句子。更何况作为读者，一两句话没搞懂，并不影响你理解整本书，但对译者来说，这就不一样了。

其次，这是一本讲英语的人写给讲英语的人的书，所以同很多要照顾非英语读者的技术文档不同，它在用词，句式方面非常随意。英语读者会很欣赏这一点，但是对外国读者来说，这就是负担了。

再有，Bruce Eckel 这样的大牛人，写了 1000 多页，如果都让你读懂，他岂不是太没面子？所以，书里还有一些很有“禅意”的句子。比如那句著名的“The genesis of the computer revolution was in a machine. The genesis of our programming languages thus tends to look like that machine.”我就一直没吃准该怎么翻译。我想大概没人能吃准，说不定 Bruce 要的就是这个效果。

这是一本公认的名著，作者在技术上的造诣无可挑剔。而作为译者，我的编程能力差了很多。再加上上面讲的这些原因，使得我不得不格外的谨慎。当我重读初稿的时候，我发现需要修改的地方实在太多了。因此，我不能现在就公开全部译稿，我只能公开已经修改过的部分。不过这不是最终的版本，我还会继续修订的。

本来，我准备到 10 月份，等我修改完前 7 章之后再公开。但是，我发现我又有点要放弃了，因此我决定给自己一点压力，现在就公开。以后，我将修改完一章就公开一章，请关注 www.wgqqh.com/shhgs/tij.html。

如果你觉得好，请给告诉我，你的鼓励是我工作的动力；如果你觉得不好，那就更应该告诉我了，我会参考你的意见作修改的。我希望能通过这种方法，译出一本配得上原著的书。

shhgs
2003 年 9 月 8 日


www.wgqqh.com/shhgs/tij.html
email:shhgs@sohu.com

# 8: 接口与内部类

接口(interface)和内部类(inner class)提供了一种更为复杂的组织和控制系统中对象的方法。

比方说，C++就没有这种机制，不过聪明的程序员还是能模拟出这种效果。Java 之所以会有这个特性，是因为设计人员认为它非常重要，语言应该直接用关键词提供支持。

你已经在第 7 章学过了 **abstract** 关键词，它能让你在类里创建一个或多个没有定义的方法——你给出了接口，但是留一部分没做定义，这部分要由它的继承类来定义。而 **interface** 关键词则创建了一种完全抽象的，根本不提供实现的类。你会看到，**interface** 不仅是一种抽象类的极端表现形式，它还是一种能让你将一个对象上传到多个基类的手段，因此它提供了类似 C++的“多重继承(multiple inheritance)”的功能。

初看起来，内部类像是一种简单的隐藏代码的机制：你只是把一个类放到另一个类里。但是，你将会看到，内部类可没这么简单——它还知道该怎样同宿主类(surrounding class)打交道——因此，即使很多人还不熟悉内部类，你还是能用它写出更为优雅清晰的代码。你得过一段时间才能把内部类熟练地运用到设计之中。

## 接口(interface)

**interface** 关键词进一步强化了 **abstract** 的概念。你可以把它想像成“纯的”**abstract** 类。它能让开发人员定义类的形式：方法名，参数列表，返回值的类型，但是却没有方法的正文。**interface** 也可以包含数据成员，但是它天生就是 **static** 和 **final** 的。**interface** 只提供形式，不谈实现。

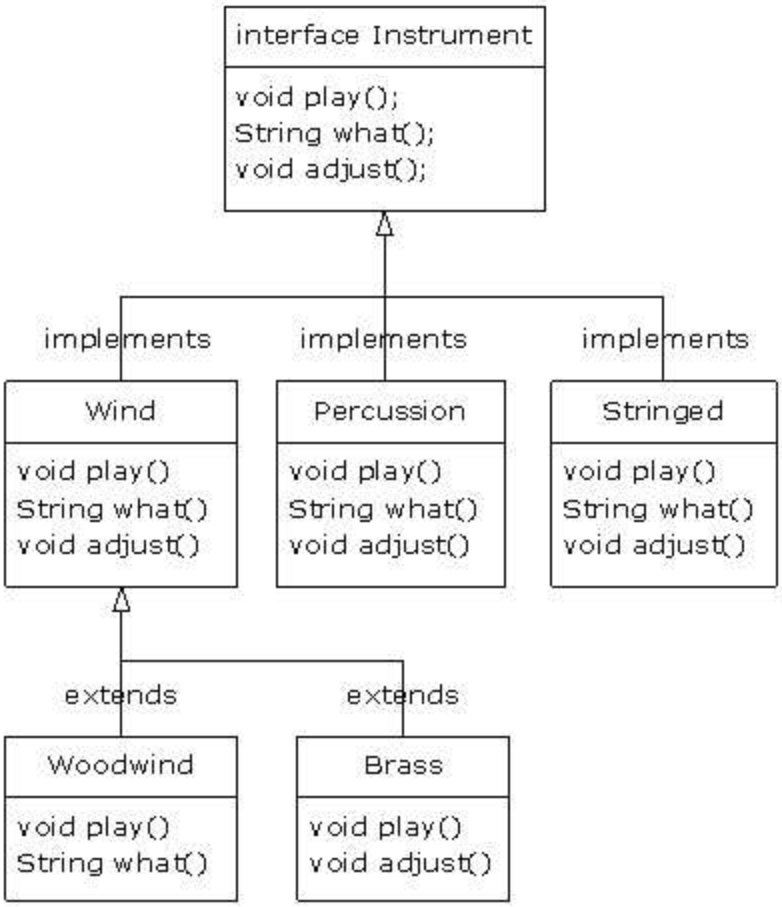
**interface** 的意思是“所有‘实现’这个接口的类都应该长这个样子。”因此任何程序，只要用到了这个 **interface** 就都知道它有那些方法可供调用了，仅此而已。因此，**interface** 会被用作定义类之间的“协议(protocol)”。(有些面向对象的语言真的用 *protocol* 关键词来作这件事。)

要想创建 **interface**，只要把 **class** 关键词换成 **interface** 就行了。跟类一样，你可以在 **interface** 关键词前面加上 **public**(只有保存在同名文件里的 **interface** 才可以加)，或者把它空着，留给它 package 权限，这样它就只能用于同一个 package 了。

要创建一个实现了某个(或者某组)**interface** 的类，就必须使用 **implements** 关键词。它的意思是，“**interface** 要告诉你‘类长什么


www.wgqqh.com/shhgs/tij.html
email:shhgs@sohu.com

样子’，但是现在我要告诉你‘它是怎样『工作』的’。”除此之外，它同继承没什么两样。还是以乐器为例，下面的图演示了这种的关系：




可以从 **Woodwind** 和 **Brass** 看出，类一旦实现了某个 **interface**，它就变成了一个可以再继承下去的普通类了。

你可以把 **interface** 里的方法声明成 **public** 的，但是即便不讲，它们也是 **public** 的。所以当你 **implements** 一个 **interface** 的时候，你必须把这个 **interface** 的方法定义成 **public** 的。如果你不这么做，那它就会变成 package 权限的，这样经过继承，这些方法的访问权限就会受到限制，而这是 Java 的编译器所不允许的。

可以从修改后的 **Instrument** 例程中看到这一点。注意，编译器只允许你在 **interface** 里面声明方法。此外，虽然 **Instrument** 的方法都没有被声明成 **public** 的，但是它们自动都是 **public** 的：

```
//: c08:music5:Music5.java
// Interfaces.
package c08.music5;
import com.bruceeckel.simpletest.*;
import c07.music.Note;

interface Instrument {
```


www.wgqqh.com/shhgs/tij.html
email:shhgs@sohu.com

```
  // Compile-time constant:
  int I = 5; // static & final
  // Cannot have method definitions:
  void play(Note n); // Automatically public
  String what();
  void adjust();
}

class Wind implements Instrument {
  public void play(Note n) {
    System.out.println("Wind.play() " + n);
  }
  public String what() { return "Wind"; }
  public void adjust() {}
}

class Percussion implements Instrument {
  public void play(Note n) {
    System.out.println("Percussion.play() " + n);
  }
  public String what() { return "Percussion"; }
  public void adjust() {}
}

class Stringed implements Instrument {
  public void play(Note n) {
    System.out.println("Stringed.play() " + n);
  }
  public String what() { return "Stringed"; }
  public void adjust() {}
}

class Brass extends Wind {
  public void play(Note n) {
    System.out.println("Brass.play() " + n);
  }
  public void adjust() {
    System.out.println("Brass.adjust()");
  }
}

class Woodwind extends Wind {
  public void play(Note n) {
    System.out.println("Woodwind.play() " + n);
  }
  public String what() { return "Woodwind"; }
}

public class Music5 {
  private static Test monitor = new Test();
  // Doesn't care about type, so new types
  // added to the system still work right:
  static void tune(Instrument i) {
    // ...
    i.play(Note.MIDDLE_C);
  }
  static void tuneAll(Instrument[] e) {
    for(int i = 0; i < e.length; i++)
      tune(e[i]);
  }
```


www.wgqqh.com/shhgs/tij.html
email:shhgs@sohu.com

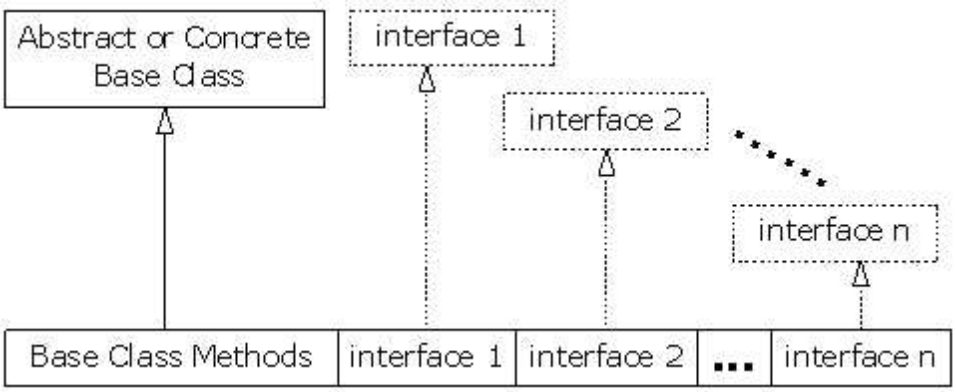
```
  public static void main(String[] args) {
    // Upcasting during addition to the array:
    Instrument[] orchestra = {
      new Wind(),
      new Percussion(),
      new Stringed(),
      new Brass(),
      new Woodwind()
    };
    tuneAll(orchestra);
    monitor.expect(new String[] {
      "Wind.play() Middle C",
      "Percussion.play() Middle C",
      "Stringed.play() Middle C",
      "Brass.play() Middle C",
      "Woodwind.play() Middle C"
    });
  }
} ///:~
```

其它代码的工作方式没变。不论是把它上传到一个叫 **Instrument** 的“普通”类，还是一个叫 **Instrument** 的 **abstract** 类，还是一个叫 **Instrument** 的 **interface**，它的工作方式都是一样的。实际上，你根本没法从 **tune( )**来判断，**Instrument** 到底是“普通”类，还是 **abstract** 类，或 **interface**。这就是它的本意：它让程序员自己去选择要在那个级别上控制对象的创建和使用。

## Java 的“多重继承”

**interface** 不仅仅是一种“更纯”的 **abstract** 类。它还有更高一层的目的。由于 **interface** 不带任何“实现”——也就是说 **interface** 和内存无关——因此不会有谁去组绕 **interface** 之间的结合。这一点非常重要，因为有时你会遇到“**x** 既是 **a** 又是 **b**，而且还是 **c**”的情况。在 C++中，这种“将多个类的接口结合在一起”的行为被称作“多重继承(*multiple inheritance*)”，但是由于每个类又都有它自己的实现，而这会带来很多“甩都甩不掉”的问题。Java 能让你作同样的事情，但是这时只有一个类可以有实现，因此当你合并 Java 接口的时候，就不会有这种问题了：


www.wgqqh.com/shhgs/tij.html
email:shhgs@sohu.com




Java 并不强制你一定要去继承 **abstract** 还是“具体”的类(就是不带 **abstract** 方法的类)，但是你只能继承一个非 **interface** 的类。所有别的基类元素(base elements)都必须是 **interface**。你得把所有的接口名字都放在 **implements** 关键词后面，用逗号把它们分开。你可以根据需要，实现任意多个 **interface**；也可以将这个类上传至任何一个 **interface**。下面这段程序演示了如何将一个具体的类同几个 **interface** 合并起来，创建一个新的类：

```
//: c08:Adventure.java
// Multiple interfaces.

interface CanFight {
  void fight();
}

interface CanSwim {
  void swim();
}

interface CanFly {
  void fly();
}

class ActionCharacter {
  public void fight() {}
}

class Hero extends ActionCharacter
    implements CanFight, CanSwim, CanFly {
  public void swim() {}
  public void fly() {}
}

public class Adventure {
  public static void t(CanFight x) { x.fight(); }
  public static void u(CanSwim x) { x.swim(); }
  public static void v(CanFly x) { x.fly(); }
  public static void w(ActionCharacter x)
{ x.fight(); }
  public static void main(String[] args) {
    Hero h = new Hero();
    t(h); // Treat it as a CanFight
    u(h); // Treat it as a CanSwim
```


www.wgqqh.com/shhgs/tij.html
email:shhgs@sohu.com

```
    v(h); // Treat it as a CanFly
    w(h); // Treat it as an ActionCharacter
  }
} ///:~
```

可以看到，**Hero** 合并了具体的 **ActionCharacter** 类，以及 **CanFight**，**CanSwim** 和 **CanFly** 接口。当你用这种方式合并实体类(concrete class)和接口的时候，必须将实体类放在前面，然后才是接口。(否则编译器就会报错。)

注意 **ActionCharacter** 类的 **fight( )**方法。它的特征与 **interface CanFight** 的 **fight( )**方法完全相同，但是 **Hero** 没有提供 **fight( )**的定义。**interface** 的规则是这样的，你可以继承它(马上就会说到)，但是继承下来的还是 **interface**。如果你想创建一个新类型的对象，那么这个类型就必须是类，而且还得提供所有定义。尽管 **Hero** 没有明确的提供 **fight( )**的定义，但是 **ActionCharacter** 提供了，所以 **Hero** 能自动获得这个方法，并且能创建对象了。

**Adventure** 类有四个拿接口和实体类作参数的方法。创建出来的 **Hero** 对象可以被传给其中任何一个方法，也就是说它被依次上传给了各个 **interface**。这个过程不需要程序员编写特别的代码，这一切要归功于接口在 Java 中的设计。

上述程序告诉我们接口的真正目的：能够上传到多个基本类型(base type)。然而，使用接口的第二个理由，实际上是和“把 **abstract** 类用做基类”完全相同的：就是要禁止客户程序员去创建这个类的对象，并且重申“这只是一个接口”。这就带来了一个问题：到底是用 **interface**，还是用 **abstract** 类？**interface** 既给了你 **abstract** 类的好处，又给了你 **interface** 的好处，因此只要基类的设计里面可以不包括方法和成员变量的定义，你就应该优先使用 **interface**。实际上，如果你知道这样东西可能会是基类的话，你就应该优先考虑把它做成 **interface**，只有在不得不定义方法或成员变量的情况下，你才能把它改成 **abstract** 类，或者根据需要改成实体类。

### 合并接口时的名字冲突

实现多个接口的时候可能会遇到一些小问题。在上述例程中，**CanFight** 和 **ActionCharacter** 都有一个一摸一样的 **void fight( )**方法。这里没有问题，因为它们使用的是同一个方法。但是如果不是呢？下面就是一个例子：

```
//: c08:InterfaceCollision.java

interface I1 { void f(); }
interface I2 { int f(int i); }
```


www.wgqqh.com/shhgs/tij.html
email:shhgs@sohu.com

```
interface I3 { int f(); }
class C { public int f() { return 1; } }

class C2 implements I1, I2 {
  public void f() {}
  public int f(int i) { return 1; } // overloaded
}

class C3 extends C implements I2 {
  public int f(int i) { return 1; } // overloaded
}

class C4 extends C implements I3 {
  // Identical, no problem:
  public int f() { return 1; }
}

// Methods differ only by return type:
//! class C5 extends C implements I1 {}
//! interface I4 extends I1, I3 {} ///:~
```

这个难题要归因于覆写、实现和重载的不期而遇，以及“不能仅通过返回值来辨别重载的方法”。如果把最后两行的注释去掉，就会出现如下的错误信息：

*InterfaceCollision.java:23: f( ) in C cannot implement f( ) in I1; attempting to use incompatible return type*
*found : int*
*required: void*
*InterfaceCollision.java:24: interfaces I3 and I1 are incompatible; both define f( ), but with different return type*

而且在要合并的接口里面放上同名方法，通常也会破坏程序的可读性。所以别这么做。

## 用继承扩展 interface

你可以用继承，往 **interface** 里面添加新的方法，也可以用继承把多个 **interface** 合并成一个新的 **interface**。在这两种情况下，你所得到的都只是一个新的 interface，就像下面这样：

```
//: c08:HorrorShow.java
// Extending an interface with inheritance.

interface Monster {
  void menace();
}

interface DangerousMonster extends Monster {
  void destroy();
}
```


www.wgqqh.com/shhgs/tij.html
email:shhgs@sohu.com

```
interface Lethal {
  void kill();
}

class DragonZilla implements DangerousMonster {
  public void menace() {}
  public void destroy() {}
}

interface Vampire extends DangerousMonster, Lethal {
  void drinkBlood();
}

class VeryBadVampire implements Vampire {
  public void menace() {}
  public void destroy() {}
  public void kill() {}
  public void drinkBlood() {}
}

public class HorrorShow {
  static void u(Monster b) { b.menace(); }
  static void v(DangerousMonster d) {
    d.menace();
    d.destroy();
  }
  static void w(Lethal l) { l.kill(); }
  public static void main(String[] args) {
    DangerousMonster barney = new DragonZilla();
    u(barney);
    v(barney);
    Vampire vlad = new VeryBadVampire();
    u(vlad);
    v(vlad);
    w(vlad);
  }
} ///:~
```

**DangerousMonster** 只是对 **Monster** 做了一点扩展，然后生成一个新的 **interface**。**DragonZilla** 则实现了这个接口。

**Vampire** 的语法是“接口继承(inheriting interfaces)”所独有的。通常情况下， **extends** 只能用于类，但是由于一个 **interface** 可以由多个接口拼接而成，因此创建新的 **interface** 的时候可以用 **extends** 来表示其多个“基接口(base interfaces)”。正如你所看到的，**interface** 的名字要由逗号分隔。

## 常量的分组

由于 **interface** 的数据成员自动就是 **static** 和 **final** 的，因此 **interface** 是一种非常方便的，创建一组常量值的工具。这点同 C 和 C++的 **enum** 很相似。例如：


www.wgqqh.com/shhgs/tij.html
email:shhgs@sohu.com

```
//: c08:Months.java
// Using interfaces to create groups of constants.
package c08;

public interface Months {
  int
    JANUARY = 1, FEBRUARY = 2, MARCH = 3,
    APRIL = 4, MAY = 5, JUNE = 6, JULY = 7,
    AUGUST = 8, SEPTEMBER = 9, OCTOBER = 10,
    NOVEMBER = 11, DECEMBER = 12;
} ///:~
```

注意一下，Java 的编程风格是，用全部大写字母(用下划线分隔同一个标识符里的各个单词)来表示，用常量进行初始化的 **static final** 变量。

**interface** 的数据成员自动就是 **public** 的，因此就不必再注明了。

你可以像对待别的 package 那样，用 **import c08.***或者 **c08.Months** 把它引进来，这样就能在这个 package 的外面用 **Months.JANUARY** 之类的表达式来使用这些常量了。当然，你得到的是一个 **int**，因此它没有像 C++的 **enum** 那样的类型安全，但是这种(很常见的)手法要比直接在程序里面用数字要好得多。(这种方法通常被成为使用“神奇数字”，并且使得代码的维护变得非常困难。)

如果你确实需要额外的类型安全，可以像这样创建一个类：[33]

```
//: c08:Month.java
// A more robust enumeration system.
package c08;
import com.bruceeckel.simpletest.*;

public final class Month {
  private static Test monitor = new Test();
  private String name;
  private Month(String nm) { name = nm; }
  public String toString() { return name; }
  public static final Month
    JAN = new Month("January"),
    FEB = new Month("February"),
    MAR = new Month("March"),
    APR = new Month("April"),
    MAY = new Month("May"),
    JUN = new Month("June"),
    JUL = new Month("July"),
    AUG = new Month("August"),
    SEP = new Month("September"),
    OCT = new Month("October"),
    NOV = new Month("November"),
    DEC = new Month("December");
  public static final Month[] month =  {
    JAN, FEB, MAR, APR, MAY, JUN,
    JUL, AUG, SEP, OCT, NOV, DEC
  };
```


www.wgqqh.com/shhgs/tij.html
email:shhgs@sohu.com

```
  public static final Month number(int ord) {
    return month[ord - 1];
  }
  public static void main(String[] args) {
    Month m = Month.JAN;
    System.out.println(m);
    m = Month.number(12);
    System.out.println(m);
    System.out.println(m == Month.DEC);
    System.out.println(m.equals(Month.DEC));
    System.out.println(Month.month[3]);
    monitor.expect(new String[] {
      "January",
      "December",
      "true",
      "true",
      "April"
    });
  }
} ///:~
```

**Month** 是一个带 **private** 构造函数的 **final** 类，因此谁也不能继承它，或者创建它的实例。所有实例都是由类自己创建的，并且都是 **final static** 的：**JAN**，**FEB**，**MAR**，等等。这些对象也被用于 **month** 数组。这是一个供你遍历的 **Month** 对象的数组。你可以传一个数字给 **number( )**方法，以选取相应月份的 **Month** 对象。从 **main( )** 可以看出，这么做是类型安全的；**m** 是一个 **Month** 的对象，所以它只能被赋予 **Month**。再前一个例子中，**Month.java** 所返回的是一个 **int** 的值，因此可能会给这个表示月份的 **int** 变量赋上其它的值，因此不是非常安全。

正如在 **main( )**的最后几行所看到的，这里你还能互换地使用**==** 和 **equals( )**。之所以能这么做，是因为 **Month** 的每个值只能有一个实例。到第 11 章，你还会学到一种新的定义类的方法，用这种类创建的对象能够进行相互比较。

此外 **java.util.Calendar** 里面有一个 month 的成员。

Apache 的 Jakarta Commons 项目下还包含了一些创建枚举类型的工具，它的功能与上述例程相似，但是用起来没那么麻烦。参见 *http://jakarta.apache.org/commons* 下面的“lang”，这个包是 **org.apache.commons.lang.enum**。这个项目里面还有很多别的，可能会非常有用的类库。

## 初始化接口中的数据成员


www.wgqqh.com/shhgs/tij.html
email:shhgs@sohu.com

接口所定义的数据都自动是 **static** 和 **final** 的。它们不能是“空白的 final 数据成员”，但是可以用非常量的表达式来对它们进行初始化。例如：

```
//: c08:RandVals.java
// Initializing interface fields with
// non-constant initializers.
import java.util.*;

public interface RandVals {
  Random rand = new Random();
  int randomInt = rand.nextInt(10);
  long randomLong = rand.nextLong() * 10;
  float randomFloat = rand.nextLong() * 10;
  double randomDouble = rand.nextDouble() * 10;
} ///:~
```

由于数据成员都是 **static** 的，因此装载类的时候，也就是第一次访问这些数据成员的时候进行初始化。下面就是一个简单的测试：

```
//: c08:TestRandVals.java
import com.bruceeckel.simpletest.*;

public class TestRandVals {
  private static Test monitor = new Test();
  public static void main(String[] args) {
    System.out.println(RandVals.randomInt);
    System.out.println(RandVals.randomLong);
    System.out.println(RandVals.randomFloat);
    System.out.println(RandVals.randomDouble);
    monitor.expect(new String[] {
      "%% -?\\d+",
      "%% -?\\d+",
      "%% -?\\d\\.\\d+E?-?\\d+",
      "%% -?\\d\\.\\d+E?-?\\d+"
    });
  }
} ///:~
```

当然，这些数据都不算是接口的组成部分，相反它们保存在这个接口的静态存储区内。

## 接口的嵌套

接口既可以嵌套在类里，也可以嵌套在接口里面。[34]这一点揭示了许多非常有趣的特点：

```
//: c08:nesting:NestingInterfaces.java
package c08.nesting;
```


www.wgqqh.com/shhgs/tij.html
email:shhgs@sohu.com

```
class A {
  interface B {
    void f();
  }
  public class BImp implements B {
    public void f() {}
  }
  private class BImp2 implements B {
    public void f() {}
  }
  public interface C {
    void f();
  }
  class CImp implements C {
    public void f() {}
  }
  private class CImp2 implements C {
    public void f() {}
  }
  private interface D {
    void f();
  }
  private class DImp implements D {
    public void f() {}
  }
  public class DImp2 implements D {
    public void f() {}
  }
  public D getD() { return new DImp2(); }
  private D dRef;
  public void receiveD(D d) {
    dRef = d;
    dRef.f();
  }
}

interface E {
  interface G {
    void f();
  }
  // Redundant "public":
  public interface H {
    void f();
  }
  void g();
  // Cannot be private within an interface:
  //! private interface I {}
}

public class NestingInterfaces {
  public class BImp implements A.B {
    public void f() {}
  }
  class CImp implements A.C {
    public void f() {}
  }
  // Cannot implement a private interface except
  // within that interface's defining class:
  //! class DImp implements A.D {
  //!   public void f() {}
```


www.wgqqh.com/shhgs/tij.html
email:shhgs@sohu.com

```
  //! }
  class EImp implements E {
    public void g() {}
  }
  class EGImp implements E.G {
    public void f() {}
  }
  class EImp2 implements E {
    public void g() {}
    class EG implements E.G {
      public void f() {}
    }
  }
  public static void main(String[] args) {
    A a = new A();
    // Can't access A.D:
    //! A.D ad = a.getD();
    // Doesn't return anything but A.D:
    //! A.DImp2 di2 = a.getD();
    // Cannot access a member of the interface:
    //! a.getD().f();
    // Only another A can do anything with getD():
    A a2 = new A();
    a2.receiveD(a.getD());
  }
} ///:~
```

在类里嵌套接口的语法还是比较明显的。跟没被嵌套在类里的接口一样，它也可以是 **public** 或 package 访问权限的。而且你还可以看到，**public** 和 package 权限的接口都被实现成 **public**，package 权限，甚至 **private** 的“嵌套类(nested classes)”。

正如 **A.D** 所演示的，作为一种新的技巧，接口也可以是 **private** 的(这种语法既可用于“嵌套接口『nested interfaces』”，也可用于“嵌套类『nested classes』”)。那么 **private** 的嵌套接口又有什么好处呢？可能你会认为，它只能被实现成 **private** 的内部类，就像 **DImp** 那样，但是 **A.DImp2** 告诉我们，它也能被实现成 **public** 的类。但是 **A.DImp2** 只能被当作它自己的类型来用。你不能透露任何“它实现了一个 **private** 接口”的信息，因此实现 **private** 接口就成了一种“强制你去定义那个接口的方法，但是又不让你添加任何类型信息(也就是说不允许上传)”的手段了。

**getD( )**方法揭示了 **private** 接口的更深层次的困境：它是一个 **public** 方法，但却会返回一个 **private** 接口的 reference。这个返回值又该怎么用呢？可以看到，**main( )**作了好几次尝试，试图利用这个返回值，但是都失败了。唯一行得通的办法就是，把这个返回值交给一个有权用它的对象——这就是通过 **A** 的 **receivedD( )**方法。


www.wgqqh.com/shhgs/tij.html
email:shhgs@sohu.com

接口 **E** 揭示了，接口与接口也可以相互嵌套。不过接口的规则仍然有效——特别是接口的所有元素都必须是 **public** 的这条，因此嵌套在接口中的接口也都自动是 **public** 的，它们不能是 **private** 的。

**NestingInterfaces** 演示了各种实现嵌套接口的办法。特别要注意的是，实现接口的时候，不一定要实现嵌套在里面的接口。同样 **private** 接口只能在定义它的类里实现。

初看起来，这些特性好像只是为“语法的一致性”服务的，但是根据我的经验，一旦你理解了一种特性，自然会找到运用它的地方。

# 内部类

在一个类里定义另一个类是完全可以的。这被称为“内部类(*inner class*)”。内部类是一种非常有价值的特性，它能让你在逻辑上将相互从属的类组织起来，并且在类的内部控制访问权限。但是切记，内部类和合成是截然不同的，这一点非常重要。

虽然你正在学习内部类，但却不清楚为什么要有它。等到这一节的末尾，等我们讲完内部类的语法和语义之后，你就会看到一些“能告诉你内部类有什么好处”的例子了。

创建内部类的方法同你想像的完全相同——将类的定义放到它的宿主类(surrounding class)：

```
//: c08:Parcel1.java
// Creating inner classes.

public class Parcel1 {
  class Contents {
    private int i = 11;
    public int value() { return i; }
  }
  class Destination {
    private String label;
    Destination(String whereTo) {
      label = whereTo;
    }
    String readLabel() { return label; }
  }
  // Using inner classes looks just like
  // using any other class, within Parcel1:
  public void ship(String dest) {
    Contents c = new Contents();
    Destination d = new Destination(dest);
    System.out.println(d.readLabel());
  }
  public static void main(String[] args) {
    Parcel1 p = new Parcel1();
    p.ship("Tanzania");
  }
```


www.wgqqh.com/shhgs/tij.html
email:shhgs@sohu.com

```
} ///:~
```

仅就 **ship( )**而言，内部类的用法同其它类没什么两样。实际上唯一的区别就是，类的名字被嵌套在 **Parcel1** 里面了。不过，过一会你就会看到，这并不是唯一的区别。

比较常见的，还是让宿主类提供一个会返回内部类的 reference 的方法，就像这样：

```
//: c08:Parcel2.java
// Returning a reference to an inner class.

public class Parcel2 {
  class Contents {
    private int i = 11;
    public int value() { return i; }
  }
  class Destination {
    private String label;
    Destination(String whereTo) {
      label = whereTo;
    }
    String readLabel() { return label; }
  }
  public Destination to(String s) {
    return new Destination(s);
  }
  public Contents cont() {
    return new Contents();
  }
  public void ship(String dest) {
    Contents c = cont();
    Destination d = to(dest);
    System.out.println(d.readLabel());
  }
  public static void main(String[] args) {
    Parcel2 p = new Parcel2();
    p.ship("Tanzania");
    Parcel2 q = new Parcel2();
    // Defining references to inner classes:
    Parcel2.Contents c = q.cont();
    Parcel2.Destination d = q.to("Borneo");
  }
} ///:~
```

除非是在“宿主类(outer class)”的非 **static** 方法里面，否则无论你在那里创建内部类的对象，都必须用方μ1 2


www.wgqqh.com/shhgs/tij.html
email:shhgs@sohu.com

```
//: c08:Callbacks.java
// Using inner classes for callbacks
import com.bruceeckel.simpletest.*;

interface Incrementable {
  void increment();
}

// Very simple to just implement the interface:
class Callee1 implements Incrementable {
  private int i = 0;
  public void increment() {
    i++;
    System.out.println(i);
  }
}

class MyIncrement {
  void increment() {
    System.out.println("Other operation");
  }
  static void f(MyIncrement mi) { mi.increment(); }
}

// If your class must implement increment() in
// some other way, you must use an inner class:
class Callee2 extends MyIncrement {
  private int i = 0;
  private void incr() {
    i++;
    System.out.println(i);
  }
  private class Closure implements Incrementable {
    public void increment() { incr(); }
  }
  Incrementable getCallbackReference() {
    return new Closure();
  }
}

class Caller {
  private Incrementable callbackReference;
  Caller(Incrementable cbh) { callbackReference =
cbh; }
  void go() { callbackReference.increment(); }
}

public class Callbacks {
  private static Test monitor = new Test();
  public static void main(String[] args) {
    Callee1 c1 = new Callee1();
    Callee2 c2 = new Callee2();
    MyIncrement.f(c2);
    Caller caller1 = new Caller(c1);
    Caller caller2 = new
Caller(c2.getCallbackReference());
    caller1.go();
    caller1.go();
    caller2.go();
    caller2.go();
```


www.wgqqh.com/shhgs/tij.html
email:shhgs@sohu.com

```
        monitor.expect(new String[] {
          "Other operation",
          "1",
          "2",
          "1",
          "2"
        });
      }
    } ///:~
```

这段程序还进一步揭示了，“让宿主类去实现接口”和“交给内部类去做”之间的区别。从编程的角度来说，很明显，**Callee1** 更简单一些。**Callee2** 继承了 **MyIncrement**，而 **MyIncrement** 已经包括了一个它自己的 **increment( )**，而且这个方法的功能同 **Incrementable** 接口所定义的毫不相关。所以 **Callee2** 继承 **MyIncrement** 以后，就不能靠实现 **Incrementable** 接口来覆写 **increment( )**了，这就逼着你只能使用内部类来提供一个独立的实现了。此外还有一点值得注意，创建内部类的时候，别去扩展或者修改宿主类的接口。

注意，除了 **getCallbackReference( )**之外， **Callee2** 里的所有东西都是 **private** 的。**interface Incrementable** 非常重要，它是维系类与外部世界的纽带。你可以从这里看到， **interface** 是如何将接口与实现分隔开来的。

内部类 **Closure** 实现了 **Incrementable**，以此提供了一个能返回 **Caller2** 的钩子——但是这是一个安全的钩子。无论谁得到了 **Incrementable** 的 reference 都能，当然也只能调用 **increment( )**，除此之外什么都干不了(不像指针，用起来就没谱了)。

**Caller** 的构造函数需要一个 **Incrementable** 的 reference(虽然获取“回调 reference”的行为可能会发生在任何时候)，然后过些时候，用这个 reference 来“回调” **Callee** 类。

回调的价值在于其灵活性；你可以在程序运行的时候动态地选择应该调用哪个方法。到了第 14 章，这点就能看得更清楚了。为实现 GUI 功能，回调会到处都是。

## 内部类与控制框架 (Inner classes & control frameworks)

内部类还有一种更实际的用法，我把它称为“控制框架(*control framework*)”。

“应用程序框架(*application framework*)”是一个或一组为解决某种特定类型的问题而设计的类。如果想使用应用程序框架，通常情况下只要继


www.wgqqh.com/shhgs/tij.html
email:shhgs@sohu.com

承其中的一个或多个类，再覆写某些方法就可以了。应用程序框架为你提供了一套解决问题的通用方案，而你只要覆写方法就可以根据你的特殊要求定制这个方案了 (这就是一种“模板方法(Template Method)”设计模式；见 *www.BruceEckel.com* 的 *Thinking in Patterns(with Java)*)。控制框架是应用程序框架中的一种，主要用于响应事件；如果系统的首要任务就是对事件作出响应，那么它就被称为“事件驱动系统(*event-driven system*)”。图形用户界面(GUI)是创建应用程序时要解决的最棘手的问题之一，它差不多就是完全由时间驱动的。正如你将在第 14 章看到的，Java 的 Swing 类库就是一个控制框架。它通过频繁的使用内部类，非常潇洒地解决了 GUI 的难题。

为了看看控制框架究竟是怎样用内部类来创建和使用的，我们想像一下这样一个控制框架，其任务是：只要事件“准备完毕”，它就执行那些事件。尽管“准备完毕”可以有很多意思，但这里只是指时间。接下来是一个没有包含“到底在控制什么”的信息的控制框架。这个信息会在实现“模板方法”的时候通过继承获取。

首先是描述控制事件的接口。它不是一个 **interface**，而是一个 **abstract** 类，这是因为默认的行为是根据时间来进行控制。因此，这个类里已经有了一些实现：

```
//: c08:controller:Event.java
// The common methods for any control event.
package c08.controller;

public abstract class Event {
  private long eventTime;
  protected final long delayTime;
  public Event(long delayTime) {
    this.delayTime = delayTime;
    start();
  }
  public void start() { // Allows restarting
    eventTime = System.currentTimeMillis() +
delayTime;
  }
  public boolean ready() {
    return System.currentTimeMillis() >= eventTime;
  }
  public abstract void action();
} ///:~
```

当你想让 **Event** 运行的时候，它的构造函数会获取当前时间(对象创建的那个时刻)，然后调用 **start( )**。**start( )**的任务是用当前时间加上延时算出什么时候启动这个事件。**start( )**没有被放入构造函数，相反它被单独列成一个方法，这样事件完成之后，你就可以再次启动计时器以重新使用这个 **Event** 对象。比方说，如果你想重复这个事件，只要写一个 **action( )**方法，然后让它直接调用 **start( )**就行了。


www.wgqqh.com/shhgs/tij.html
email:shhgs@sohu.com

**ready( )**会告诉你什么时候可以运行 **action( )**方法了。当然，派生类可以覆写 **ready( )**，这样就能把 **Event** 改造成不是基于时间的了。

下面这段程序是一个货真价实的，可以控制和发出事件控制框架。**Event** 对象被保存在一个被称为 **ArrayList** 的容器对象中。我们要到第 11 章才开始系统地学习 **ArrayList**，现在你只要知道 **add( )**会把 **Object** 添加到 **ArrayList** 的最后，**size( )**会告诉你 **ArrayList** 里面有多少对象，**get( )**可以根据下标从 **ArrayList** 里面把元素提取出来，而 **remove( )**会根据你给出下标把元素从 **ArrayList** 里面删除。

```
//: c08:controller:Controller.java
// With Event, the generic framework for control
systems.
package c08.controller;
import java.util.*;

public class Controller {
  // An object from java.util to hold Event objects:
  private List eventList = new ArrayList();
  public void addEvent(Event c) { eventList.add(c); }
  public void run() {
    while(eventList.size() > 0) {
      for(int i = 0; i < eventList.size(); i++) {
        Event e = (Event)eventList.get(i);
        if(e.ready()) {
          System.out.println(e);
          e.action();
          eventList.remove(i);
        }
      }
    }
  }
} ///:~
```

**run( )** 方法会遍历整个 **eventList**，以寻找已经 **ready( )**的 **Event** 对象。一旦它找到一个 **ready( )**的对象，它会用 **toString( )**方法把它打印出来，然后在调用这个对象的 **action( )**方法，最后再在列表里面把这个 **Event** 删除。

注意，到目前为止，你还不知道要让 **Event** 干些什么。而这就是设计的要旨——怎样才能“将会变和不会变的东西分开来”。或者，用我的话来说，“变化的向量(vector of change)”就是各种 **Event** 对象的不同行为，而你的任务是，创建各种 **Event** 的子类来表达不同的行为。

现在内部类入场了。它们能做两件事：

1. 在一个类里完整地实现整个控制框架，这样就把“实现”里的所有独一无二东西全都给封装起来了。内部类则用来表示各种解决具体问题所需的 **action( )**。


www.wgqqh.com/shhgs/tij.html
email:shhgs@sohu.com

2. 内部类可以让这个实现看上去不至于太过古怪，因为它还可以直接访问宿主类的所有成员。要不是这样的话，代码就会变得非常杂乱，这样到最后你肯定会去找一个替代方案的。

想想下面这个控制框架的具体实现，它是用来控制暖房的。[39]每个动作都截然不同：开灯，浇水，开或关温控器，响铃，重新启动系统。但是控制框架可以非常简单的隔离不同的代码。内部类能让你在一个类里从同一个基类，**Event**，派生出多个继承类。针对每种行为，你都可以继承一种新的 **Event** 内部类，然后把控制代码写进 **action( )**。

与典型的应用程序框架一样，**GreenhouseControls** 类是从 **Controller** 继承下来的：

```
//: c08:GreenhouseControls.java
// This produces a specific application of the
// control system, all in a single class. Inner
// classes allow you to encapsulate different
// functionality for each type of event.
import com.bruceeckel.simpletest.*;
import c08.controller.*;

public class GreenhouseControls extends Controller {
  private static Test monitor = new Test();
  private boolean light = false;
  public class LightOn extends Event {
    public LightOn(long delayTime)
{ super(delayTime); }
    public void action() {
      // Put hardware control code here to
      // physically turn on the light.
      light = true;
    }
    public String toString() { return "Light is
on"; }
  }
  public class LightOff extends Event {
    public LightOff(long delayTime)
{ super(delayTime); }
    public void action() {
      // Put hardware control code here to
      // physically turn off the light.
      light = false;
    }
    public String toString() { return "Light is
off"; }
  }
  private boolean water = false;
  public class WaterOn extends Event {
    public WaterOn(long delayTime)
{ super(delayTime); }
    public void action() {
      // Put hardware control code here.
      water = true;
    }
    public String toString() {
      return "Greenhouse water is on";
```


www.wgqqh.com/shhgs/tij.html
email:shhgs@sohu.com

```
    }
  }
  public class WaterOff extends Event {
    public WaterOff(long delayTime)
{ super(delayTime); }
    public void action() {
      // Put hardware control code here.
      water = false;
    }
    public String toString() {
      return "Greenhouse water is off";
    }
  }
  private String thermostat = "Day";
  public class ThermostatNight extends Event {
    public ThermostatNight(long delayTime) {
      super(delayTime);
    }
    public void action() {
      // Put hardware control code here.
      thermostat = "Night";
    }
    public String toString() {
      return "Thermostat on night setting";
    }
  }
  public class ThermostatDay extends Event {
    public ThermostatDay(long delayTime) {
      super(delayTime);
    }
    public void action() {
      // Put hardware control code here.
      thermostat = "Day";
    }
    public String toString() {
      return "Thermostat on day setting";
    }
  }
  // An example of an action() that inserts a
  // new one of itself into the event list:
  public class Bell extends Event {
    public Bell(long delayTime) { super(delayTime); }
    public void action() {
      addEvent(new Bell(delayTime));
    }
    public String toString() { return "Bing!"; }
  }
  public class Restart extends Event {
    private Event[] eventList;
    public Restart(long delayTime, Event[] eventList)
{
      super(delayTime);
      this.eventList = eventList;
      for(int i = 0; i < eventList.length; i++)
        addEvent(eventList[i]);
    }
    public void action() {
      for(int i = 0; i < eventList.length; i++) {
        eventList[i].start(); // Rerun each event
        addEvent(eventList[i]);
```


www.wgqqh.com/shhgs/tij.html
email:shhgs@sohu.com

```
        }
        start(); // Rerun this Event
        addEvent(this);
      }
      public String toString() {
        return "Restarting system";
      }
    }
    public class Terminate extends Event {
      public Terminate(long delayTime)
{ super(delayTime); }
      public void action() { System.exit(0); }
      public String toString() { return
"Terminating";  }
    }
} ///:~
```

注意 **light**，**water** 和 **thermostat** 都属于宿主类 **GreenhouseControls**，而内部类无须任何特别的许可和权限就可以访问这些数据成员。此外绝大多数 **action( )**方法都牵涉到了一些硬件的控制。

绝大多数的 **Event** 类看上去都很像，但是 **Bell** 和 **Restart** 有些特别。**Bell** 响过之后还会往事件列表里面再放一个新的 **Bell** 对象，因此它会响两次。注意一下，内部类是怎样模拟多重继承的：**Bell** 和 **Restart** 具备了 **Event** 的所有方法，此外它看上去还具备了宿主类 **GreenhouseControls** 的所有方法。

**Restart** 拿了一个 **Event** 对象的数组，并把它加入“控制器(controller)”。由于 **Restart( )**也是一种 **Event** 对象，因此你可以在 **Restart.action( )**里面加一个 **Restart** 对象，这样系统就能定时自动重启了。

下面这个类通过创建 **GreenhouseControls** 对象，然后再添加各类 **Event** 对象，完成了系统的配置。这就是“命令(*Command*)”设计模式的一例：

```
//: c08:GreenhouseController.java
// Configure and execute the greenhouse system.
// {Args: 5000}
import c08.controller.*;

public class GreenhouseController {
  public static void main(String[] args) {
    GreenhouseControls gc = new GreenhouseControls();
    // Instead of hard-wiring, you could parse
    // configuration information from a text file
here:
    gc.addEvent(gc.new Bell(900));
    Event[] eventList = {
      gc.new ThermostatNight(0),
```


www.wgqqh.com/shhgs/tij.html
email:shhgs@sohu.com

```
        gc.new LightOn(200),
        gc.new LightOff(400),
        gc.new WaterOn(600),
        gc.new WaterOff(800),
        gc.new ThermostatDay(1400)
      };
      gc.addEvent(gc.new Restart(2000, eventList));
      if(args.length == 1)
        gc.addEvent(
          gc.new Terminate(Integer.parseInt(args[0])));
      gc.run();
    }
} ///:~
```

这个类要对系统进行初始化，因此会添加所有必要的事件。当然更好的办法还是不写代码，而是从文件读取。(第 12 章的练习里面有一道题会要你这样修改这个程序。)如果你在命令行下提供了参数，那么它会在经过这些毫秒之后中止这个程序(这是用来测试的)。

你应该能从这个例子里欣赏到内部类的价值了吧，特别是当它被用于控制框架的时候。但是，到第 14 章你还会看到，内部类是如何被优雅地用于描述图形用户界面的操作的。到了那时，你才算真正读完本章，你才会对内部类的价值完全信服。

# 总结

相比绝大多数别的 OOP 语言，接口和内部类的概念会更为复杂。比方说 C++就没类似的概念。此外，它们还解决了 C++试图用多重继承(MI)解决的问题。然而 C++的 MI 非常难用，相比之下，Java 的接口和内部类则简单了许多。

虽然，这些特性本身还是比较简单的，但是它们具体用法就属于设计的范畴了，这点同多态性非常相似。随着时间的推移，当你再碰到要选择使用接口还是内部类，或者两个都用的时候，就会好一点了。但是讲到这里，你至少应该已经熟悉它的语法和语义了。随着你越来越多的接触这些特性，最终会完全领悟它的精髓的。

# 练习

只要付很小一笔费用就能从 *www.BruceEckel.com* 下载名为 *The Thinking in Java Annotated Solution Guide* 的电子文档，这上面有一些习题的答案。

1. 证明一下 **interface** 里面的数据成员默认就是 **static** 和 **final** 的。


www.wgqqh.com/shhgs/tij.html
email:shhgs@sohu.com

2. 在它自己的 package 里面创建一个有三个方法的 interface。然后在另一个 package 里面实现这个接口。
3. 证明一下，**interface** 里面的方法自动就是 **public** 的。
4. 找到 **c07:Sandwich.java**，创建一个名叫 **FastFood** 的接口(编几个像样一点的方法)，然后修改 **Sandwich**，让它实现 **FastFood**。
5. 创建三个 **interface**，每个都要有两个方法。再写一个继承这三个 **interface** 的 **interface**，并且再添加一个新的方法。创建一个类，让它在实现这个新的 **interface** 的同时还要继承一个实体类。接下来写四个方法，每个方法都要拿一个 **interface** 做参数。在 **main( )**里面创建这样一个对象，然后传给各个方法。
6. 修改练习 5，创建一个 abstract 类，然后让派生类继承这个类。
7. 修改 **Music5.java**，加入一个 **Playable interface**。把 **play( )**方法的声明从 **Instrument** 移到 **Playable** 里面。让派生类实现 **Playable** 接口。修改 **tune( )**，让它拿 **Playable** 而不是 **Instrument** 作参数。
8. 修改第 7 章的练习 6，把 **Rodent** 改写成 **interface**。
9. 找到 **Adventure.java**，按照其它接口的形式添加一个叫 **CanClimb** 的接口。
10. 写一个要引入并使用 **Month.java** 的程序。
11. 参考 **Month.java**，创建一个一星期有几天的枚举类。
12. 在它自己的 package 里创建一个至少包括一个方法的 **interface**。加入一个实现这个 **interface** 的 **proctected** 的内部类。在第三个 package 里，继承这个类，然后用它的方法返回这个 **protected** 内部类的对象，再在返回的过程中将它上传给 **interface**。
13. 创建一个至少有一个方法的 **interface**，然后在方法里面定义一个实现这个接口的内部类，再让它返回这个 **interface** 的 reference。修改练习 13，但是要在方法的作用域里定义这个内部类。
14. 用匿名内部类改写练习 13。
15. 修改 **HorrorShow.java**，用匿名内部类实现 **DangerousMonster** 和 **Vampire** 接口。
16. 创建一个实现 **public interface** 的 **private** 内部类。写一个会返回这个 **private** 内部类实例的 reference 的方法，把它上传到 **interface**。试试能不能进行下传，检验一下内部类是不是完全把它隐藏起来了。
17. 创建一个带非默认构造函数(带参数的构造函数)，并且没有默认构造函数(无参数构造函数)的类。再创建一个类，其中要有一个会返回第一个的类的 reference 的方法。通过创建继承第一个类的匿名内部类的方式，创建并返回对象。
18. 创建一个带 **private** 数据成员和 **private** 方法的类。创建一个带有“能修改宿主类数据，并且能调用宿主类方法”的方法的内部类。再另一个宿主类方法里面创建一个内部类对象，再调用它的方法，然后看看宿主类对象有什么反映。
19. 用匿名内部类改写练习 19。


www.wgqqh.com/shhgs/tij.html
email:shhgs@sohu.com

20. 创建一个包含嵌套类的类。在 **main( )**里面创建一个这种内部类的实例。
21. 创建一个包含嵌套类的 **interface**。实现这个 **interface**，并且创建一个这个嵌套类的实例。
22. 创建一个包含内部类的类，而这个内部类自己又包含一个内部类。用嵌套类重写一遍这个练习。注意一下编译器生成的**.class** 文件的名字。
23. 创建一个带内部类的类。在另一个类里创建一个这个内部类的实例。
24. 创建一个带非默认构造函数(需要参数的构造函数)的内部类的类。再创建一个带内部类的类，然后让这个内部类去继承第一个内部类。
25. 修复 **WindError.java** 中的问题。
26. 修改 **Sequence.java**，加一个 **getRSelector( )**方法，用它生成一种新的 **Selector interface** 的实现，这个 **Selector** 要能从后向前反过来访问这个序列。
27. 创建一个带三个方法的 **interface U**。创建一个带有“用创建匿名内部类方式返回 **U** 的 reference 的”方法的类 **A**。再创建一个包含一个 **U** 的数组的类 **B**。这个 **B** 应当有一个能将 **U** 的 reference 存进数组的方法，以及一个能将数组中的 reference (根据方法的参数) 设成 **null** 的方法，此外它还要有一个能遍历数组，并且调用 **U** 的方法的方法。用 **main( )**创建一组 **A** 对象以及一个 **B** 对象。用由 **A** 对象返回的 **U** 的 reference 填满 **B**。用 **B** 来回调 **A** 对象。从 **B** 里面删掉一些 **U** 的 reference。
28. 找到 **GreenhouseControls.java**，添加一个负责开关风扇(fan)的 **Event** 内部类。配置 **GreenHouseControls.java**，使之能使用这个新的 **Event** 对象。
29. 继承 **GreenhouseControls.java** 的 **GreenhouseControls** 类，添加一个负责开关水雾发生器的 **Event** 内部类。写一个新的 **GreenhouseController.java**，让它使用这个新的 **Event** 对象。
30. 证明一下内部类可以访问宿主类的 **private** 元素。看看反过来是不是也可以。

---

[33] Rich Hoffarth 的 email 给了我灵感。Joshua Bloch 在 *Effective Java* (Addison-Wesley, 2001)在第 21 条中对这个专题作了更深入的探讨。

[34]感谢 Martin Danner，他在培训班里问了这个问题。

[35]这种设计与 C++的“嵌套类(*nested classes*)”之间有着明显的区别。C++的嵌套类只是一种简单的名字隐藏的机制。它同宿主类的对象没有什么联系，也不存在什么默认的授权。


www.wgqqh.com/shhgs/tij.html
email:shhgs@sohu.com

[36]大致同 C++的嵌套类相似，只是 C++的类不能像 Java 那样访问 private 的成员。

[37]还是要感谢 Martin Danner。

[38] 另一方面‘**$**’是 Unix shell 的特殊字符，因此显示**.class** 文件的时候，有时会遇到问题。考虑到 Sun 是一家 Unix 厂商，这就有点说不过去了。我想也许他们根本就没想过这个问题，相反他们认为你更关心的应该还是源代码文件。

[39]由于某种原因，我总是喜欢拿这个问题举例；我早期的那本 *C++ Inside & Out* 里面就有了这个例子，但是 Java 能提供一个更为潇洒的解决方案。


www.wgqqh.com/shhgs/tij.html
email:shhgs@sohu.com